

Format strings are a way to inject variables into a string in Python. They are used to format strings and produce more human-readable outputs. There are several ways to format strings in Python:

String interpolation (f-strings)

Introduced in Python 3.6, f-strings are a new way to format strings in Python. They are prefixed with 'f' and use curly braces {} to enclose the variables that will be formatted. For example:

```
name = "John"age = 30print(f"My name is {name} and I am {age} years old.")
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
My name is John and I am 30 years old.
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



str.format()

This is another way to format strings in Python. It uses curly braces {} as placeholders for variables which are passed as arguments in the format() method. For example:

```
name = "John"age = 50print("My name is {} and I am {} years old.".format(name, age))
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
My name is John and I am 50 years old.
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



% Operator

This is one of the oldest ways to format strings in Python. It uses the % operator to replace variables in the string. For example:

```
name = "Johnathan"age = 30print("My name is %s and I am %d years old." % (name, age))
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
My name is Johnathan and I am 30 years old.
```

**Information:**

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



"My name is %s and I am %d years old.": This is a string that includes format specifiers:

- **%s:** This is a placeholder for a string.
- **%d:** This is a placeholder for an integer.

% (name, age): This is a tuple containing the variables name and age. The values of these variables will replace the placeholders in the string.

Each of these methods has its own advantages and use cases. However, f-strings are generally considered the most modern and preferred way to format strings in Python due to their readability and performance.

Additional capabilities

F-strings are also able to evaluate expressions inside the curly braces, which can be very handy. For example:

```
x = 10
y = 20
print(f"The sum of x and y is {x+y}.")
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
The sum of x and y is 30.
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



Raw String (r’')

In Python, raw strings are a powerful tool for handling textual data, especially when dealing with escape characters. By prefixing a string literal with the letter ‘r’, Python treats the string as raw, meaning it interprets backslashes as literal characters rather than escape sequences.

Consider the following examples of regular string and raw string:

Regular string:

```
regular_string = "C:\new_folder\file.txt"
print("Regular String:", regular_string)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
Regular String:  C:ew_folderile.txt
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



In the regular string `regular_string` variable, the backslashes (`\n`) are interpreted as escape sequences. Therefore, `\n` represents a newline character, which would lead to an incorrect file path representation.

Raw string:

```
raw_string = r"C:\new_folder\file.txt"print("Raw String:", raw_string)
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



This will output:

```
Raw String: C:\new_folder\file.txt
```



Information:

Pressing Tab in the current editor will insert the tab character. Toggle this behavior by pressing Control+M.



However, in the raw string `raw_string`, the backslashes are treated as literal characters. This means that `\n` is not interpreted as a newline character, but rather as two separate characters, `\` and `n`. Consequently, the file path is represented exactly as it appears.